Argustack MCP Tools Reference

Version 0.1.12 | March 2026

Complete reference for all 20 MCP tools. Each tool includes parameters, usage examples, and sample output.

Visit the project on GitHub or install from npm.

> **Note:** This documentation is actively maintained alongside the codebase. While we verify accuracy with each release, minor discrepancies may exist as features evolve. For the latest information, refer to the GitHub repository. Found an issue? Open a ticket.

Table of Contents

1. Overview
2. Jira Tools
3. Git Tools
4. GitHub Tools
5. Cross-Source Tools
6. Database Tools
7. System Tools
8. Raw SQL Queries
9. Database Schema Reference

1. Overview

After syncing your data (`argustack sync`), Claude accesses your local PostgreSQL database through these 20 tools:

#	Tool	Category	Purpose
1	`query_issues`	Jira	Search and filter issues
2	`get_issue`	Jira	Full details of one issue
3	`issue_stats`	Jira	Aggregate statistics
4	`pull_jira`	Jira	Sync latest data
5	`list_projects`	Jira	Available projects
6	`query_commits`	Git	Search commits
7	`issue_commits`	Git	Commits linked to an issue
8	`commit_stats`	Git	Commit statistics
9	`query_prs`	GitHub	Search pull requests
10	`issue_prs`	GitHub	PRs linked to an issue
11	`query_releases`	GitHub	List releases
12	`issue_timeline`	Cross	Full chronological timeline
13	`hybrid_search`	Cross	Combined text + semantic search (RRF)
14	`estimate`	Cross	Task duration prediction
15	`db_schema`	Database	Browse external DB schema
16	`db_query`	Database	Read-only SQL against external DB
17	`db_stats`	Database	External DB schema statistics
18	`workspace_info`	System	Workspace configuration
19	`switch_workspace`	System	Switch to a different workspace
20	`list_workspaces`	System	List all available workspaces

All tools are read-only (except `pull_jira` which syncs data). All data stays on localhost.

2. Jira Tools

query_issues

Search and query Jira issues. Supports full-text search, filtering, and raw SQL.

Parameters:

Parameter	Type	Required	Description
`search`	string	No	Full-text search query (e.g. "payment bug")
`project`	string	No	Filter by project key (e.g. "PROJ")
`status`	string	No	Filter by status (e.g. "Open", "In Progress", "Done")
`assignee`	string	No	Filter by assignee display name
`issue_type`	string	No	Filter by type (e.g. "Bug", "Story", "Task")
`limit`	number	No	Max results (default: 50)
`sql`	string	No	Raw SQL for advanced queries

Example prompts:

> "Search for issues about export functionality"

> "Show me all open bugs in project PROJ assigned to Sarah"

> "Find tasks related to authentication that are not Done"

Output format:

```
PROJ-123 [In Progress] Fix export duplicates (Sarah)
PROJ-456 [Open] Payment webhook timeout (John)
PROJ-789 [Done] Update login flow (Mike)

3 issues found
```

get_issue

Get complete details of a single issue including description, comments, changelogs, and custom fields.

Parameters:

Parameter	Type	Required	Description
`issue_key`	string	Yes	Issue key (e.g. "PROJ-123")

Example prompts:

> "Show me full details of PROJ-123"

> "What's the description and comments on PROJ-456?"

Output format:

```
# PROJ-123: Fix export duplicates

Type: Bug | Status: In Progress | Priority: High
Assignee: Sarah | Reporter: Mike
Labels: backend, export | Components: Export Module
Story Points: 3 | Sprint: Sprint 47
Created: 2025-12-01 | Updated: 2026-01-15

## Description
When exporting LOC Draws, duplicate entries appear...

## Custom Fields
- Target Release: v2.5.0
- Customer Impact: Medium

## Comments (3)
[2025-12-02] Mike: This affects the monthly report...
[2025-12-05] Sarah: Investigating — looks like a join issue...
[2025-12-08] Sarah: Found the root cause, fixing now...

## Recent Changes
[2025-12-01] Status: Open → In Progress (Sarah)
[2025-12-03] Priority: Medium → High (Mike)
[2025-12-05] Assignee: Mike → Sarah (Mike)
```

issue_stats

Aggregate statistics about issues — counts by status, type, project, assignee.

Parameters:

Parameter	Type	Required	Description
`project`	string	No	Filter stats by project key

Example prompts:

> "Give me a breakdown of all issues by status"

> "Show issue statistics for project PROJ"

Output format:

```
## Issue Statistics

By Status:
  Done:         342
  In Progress:   45
  Open:           28
  Blocked:         5

By Type:
  Story:    180
  Bug:      120
  Task:      95
  Sub-task:  25

By Project:
  PROJ:  250
  TEAM:  170

Top Assignees:
  Sarah:   85 issues
  John:    72 issues
  Mike:    63 issues
```

pull_jira

Sync latest issues from Jira into the local database. Supports incremental pulls.

Parameters:

Parameter	Type	Required	Description
`project`	string	No	Specific project key (e.g. "PROJ"). Omit to pull all configured projects.
`since`	string	No	Pull issues updated since this date (YYYY-MM-DD). Omit for auto-incremental.

Example prompts:

> "Pull the latest data from Jira"

> "Sync only the PROJ project from Jira"

list_projects

List all available Jira projects in the configured instance.

Parameters: None

Example prompts:

> "What Jira projects are available?"

3. Git Tools

query_commits

Search Git commits with full-text search, filtering by author/date/file, and raw SQL.

Parameters:

Parameter	Type	Required	Description
`search`	string	No	Full-text search in commit messages
`author`	string	No	Filter by author name
`since`	string	No	Commits after date (YYYY-MM-DD)
`until`	string	No	Commits before date (YYYY-MM-DD)
`file_path`	string	No	Filter by changed file path
`repo_path`	string	No	Filter by repository path
`limit`	number	No	Max results (default: 50)
`sql`	string	No	Raw SQL query

Example prompts:

> "Show me commits mentioning PROJ-123"

> "Find commits by Sarah in the last week"

> "What commits changed files in src/payments/?"

Output format:

```
abc1234 2026-01-15 Sarah: fix(export): resolve duplicate entries in LOC draws
def5678 2026-01-14 Sarah: refactor(export): extract query builder
ghi9012 2026-01-13 John: feat(payments): add webhook retry logic

3 commits found
```

issue_commits

Cross-reference: find all Git commits that mention a specific Jira issue key.

Parameters:

Parameter	Type	Required	Description
`issue_key`	string	Yes	Issue key (e.g. "PROJ-123")
`repo_path`	string	No	Filter by repository path

Example prompts:

> "What code was written for PROJ-123?"

> "Show me all commits linked to PROJ-456"

Output format:

```
## Commits for PROJ-123 (3 commits)

### abc1234 — fix(export): resolve duplicate entries
Author: Sarah | Date: 2026-01-15
Files:
  M src/export/query.ts        +12 -5
  M src/export/formatter.ts     +3 -1
  A tests/export/query.test.ts +45 -0

### def5678 — refactor(export): extract query builder
Author: Sarah | Date: 2026-01-14
Files:
  M src/export/query.ts        +28 -42
  A src/export/builder.ts      +35 -0
```

commit_stats

Aggregate statistics about Git commits — total count, top authors, most changed files.

Parameters:

Parameter	Type	Required	Description
`since`	string	No	Stats from date (YYYY-MM-DD)
`author`	string	No	Filter by author name
`repo_path`	string	No	Filter by repository path

Example prompts:

> "Show me commit statistics for the last quarter"

> "How many commits did Sarah make this month?"

Output format:

```
## Commit Statistics

Total commits: 1,247
Linked to issues: 892 (72%)

Top Authors:
  Sarah:   312 commits
  John:    285 commits
  Mike:    198 commits

Most Changed Files:
  src/api/handler.ts         142 changes
  src/auth/login.ts           98 changes
  src/export/query.ts         87 changes
```

4. GitHub Tools

query_prs

Search GitHub pull requests with full-text search, filtering, and raw SQL.

Parameters:

Parameter	Type	Required	Description
`search`	string	No	Full-text search in PR title and body
`state`	string	No	Filter: open, closed, merged
`author`	string	No	Filter by PR author
`base_ref`	string	No	Filter by base branch (e.g. "main")
`since`	string	No	PRs updated since date (YYYY-MM-DD)
`limit`	number	No	Max results (default: 50)
`sql`	string	No	Raw SQL query

Example prompts:

> "Show me all merged PRs this month"

> "Find open PRs by Sarah targeting main"

Output format:

```
#142 [merged] fix(export): resolve duplicate entries by Sarah (2026-01-15) +15/-6
#139 [merged] feat(payments): webhook retry by John (2026-01-14) +120/-45
#137 [open] refactor(auth): modernize login flow by Mike (2026-01-13) +230/-180

3 pull requests found
```

issue_prs

Cross-reference: find all GitHub PRs that mention a specific Jira issue key.

Parameters:

Parameter	Type	Required	Description
`issue_key`	string	Yes	Issue key (e.g. "PROJ-123")

Example prompts:

> "Was PROJ-123 reviewed? Who approved it?"

> "Show me PRs linked to PROJ-456"

Output format:

```
## Pull Requests for PROJ-123

### #142 — fix(export): resolve duplicate entries
State: merged | Author: Sarah | Merged: 2026-01-15
Base: main ← feature/fix-export

Reviews:
  John: APPROVED
  Mike: COMMENTED
```

query_releases

List GitHub releases with full-text search.

Parameters:

Parameter	Type	Required	Description
`search`	string	No	Full-text search in release name/body
`limit`	number	No	Max results (default: 20)

Example prompts:

> "List all releases"

> "Find the release that included payment changes"

5. Cross-Source Tools

issue_timeline

Full cross-source chronological timeline for a Jira issue. Combines changelogs, commits, and PRs.

Parameters:

Parameter	Type	Required	Description
`issue_key`	string	Yes	Issue key (e.g. "PROJ-123")

Example prompts:

> "Show me the complete timeline for PROJ-123"

> "How did PROJ-456 go from creation to merge?"

Output format:

```
## PROJ-123: Fix export duplicates
Status: Done | Type: Bug | Assignee: Sarah

## Timeline

2025-12-01 09:00  [JIRA] Created (Reporter: Mike)
2025-12-01 10:30  [JIRA] Status: Open → In Progress (Sarah)
2025-12-03 14:00  [JIRA] Priority: Medium → High (Mike)
2025-12-14 11:00  [GIT]  def5678 refactor(export): extract query builder
2025-12-15 09:30  [GIT]  abc1234 fix(export): resolve duplicate entries
2025-12-15 10:00  [PR]   #142 opened: fix(export): resolve duplicate entries
2025-12-15 14:00  [PR]   #142 reviewed: John → APPROVED
2025-12-15 15:30  [PR]   #142 merged into main
2025-12-15 16:00  [JIRA] Status: In Progress → Done (Sarah)
```

hybrid_search

Combined text + semantic search using Reciprocal Rank Fusion (RRF). Merges full-text keyword matching with AI vector similarity for the best of both worlds. Works without embeddings (text-only fallback).

Parameters:

Parameter	Type	Required	Description
`query`	string	Yes	Natural language query
`limit`	number	No	Max results (default: 10)
`threshold`	number	No	Min similarity 0-1 for vector results (default: 0.5)

Example prompts:

> "Find issues similar to 'users getting timeout on password reset'"

> "Search for anything related to data export failures"

Output format:

```
PROJ-123 [Done] Fix authentication flow hanging (3.2% | both)
PROJ-456 [Open] Credential recovery timeout (1.6% | semantic)
PROJ-789 [Done] Session expiry not handled (1.5% | text)

3 results (hybrid search)
```

The `source` column shows where the result came from: `text` (keyword match), `semantic` (vector similarity), or `both` (found by both methods — highest confidence).

Why it's powerful: Pure keyword search misses synonyms. Pure semantic search misses exact matches. Hybrid search combines both using Reciprocal Rank Fusion — results that appear in both searches rank highest. Works without OpenAI key (text-only mode), and gets better with embeddings enabled.

estimate

Predict how long a task will take for a specific developer. The most complex MCP tool.

Parameters:

Parameter	Type	Required	Description
`description`	string	Yes	Task description
`assignee`	string	Yes	Developer name
`issue_type`	string	No	Bug, Task, Story — finds same-type analogs
`components`	string[]	No	Component names for area matching
`exclude_key`	string	No	Issue key to exclude from analogs
`limit`	number	No	Similar tasks to analyze (default: 10)

Example prompts:

> "How long will it take Sarah to fix a payment export bug?"

> "Estimate: John implementing Stripe webhook integration, type Story, components Payments"

Output with effort data (Jira API + Git):

```
## Similar Tasks (8 found)
PROJ-100 Fix CSV export crash — 4.2h (Sarah, Bug)
PROJ-203 Export timeout on large datasets — 6.1h (John, Bug)
...

## Base Hours
3.8h (weighted trimmed mean, 7/8 tasks)

## Developer Coefficients
Sarah: ×0.85 no bugs, ×1.05 with bugs (based on 24 tasks, median)

## Prediction for Sarah
Without bugs: 3.8h × 0.85 = 3.2h (0.4 days)
With bugs:    3.8h × 1.05 = 4.0h (0.5 days) — bug overhead +24%
```

Output with CSV only (no effort data):

```
## Resolution Timeline (cycle time only)
Similar tasks were closed in 5–8 business days from creation.
This is lead time (backlog + dev + review), NOT active development time.

⚠ No effort tracking data available.
Connect Jira API or Git for actual work hours and per-developer predictions.
```

6. Database Tools

db_schema

Browse the schema of an external database connected to Argustack. Shows tables, columns, foreign keys, and indexes.

Prerequisites: Configure target database in `.env` (`TARGET_DB_HOST`, `TARGET_DB_USER`, `TARGET_DB_NAME`) and run `argustack sync db`.

Parameters:

Parameter	Type	Required	Description
`table`	string	No	Filter by table name (partial match)
`schema`	string	No	Filter by schema name (e.g. "public", "dbo")
`source`	string	No	Filter by source name (if multiple databases synced)

Example prompts:

> "Show me the schema of the users table"

> "What tables exist in the production database?"

> "Show all foreign keys referencing the orders table"

Output format:

```
Database Schema (42 tables)

## public.users (128KB, ~5200 rows)
  id: integer PK NOT NULL
  email: varchar NOT NULL
  name: varchar NULL
  created_at: timestamp NOT NULL DEFAULT now()
  Foreign keys:
    (none)
  Indexes:
    idx_users_email UNIQUE: (email)

## public.orders (1MB, ~25000 rows)
  id: integer PK NOT NULL
  user_id: integer NOT NULL
  total: numeric NOT NULL
  Foreign keys:
    user_id → users.id
```

db_query

Execute a read-only SQL query against the external database. Only SELECT, EXPLAIN, SHOW, DESCRIBE, and WITH+SELECT are allowed. Results limited to 1000 rows with 30s timeout.

Parameters:

Parameter	Type	Required	Description
`sql`	string	Yes	SQL query (read-only only)

Example prompts:

> "Run SELECT COUNT(*) FROM users WHERE created_at > '2026-01-01'"

> "Show me the top 10 orders by total amount"

Safety: SQL is validated before execution — INSERT, UPDATE, DELETE, DROP and other write operations are rejected. The connection uses session-level read-only mode where supported.

db_stats

Statistics about the external database schema stored in Argustack.

Parameters:

Parameter	Type	Required	Description
`source`	string	No	Filter by source name

Example prompts:

> "How big is the production database?"

> "Show me database schema statistics"

Output format:

```
Database Schema Statistics

Tables: 42
Columns: 387
Foreign keys: 28
Indexes: 65

By schema:
  public: 42 tables, ~150000 rows

Largest tables (by row count):
  events: ~85000 rows (12MB)
  orders: ~25000 rows (1MB)
  users: ~5200 rows (128KB)
```

7. System Tools

workspace_info

Show current workspace configuration — sources, paths, database connection.

Parameters: None

Example prompts:

> "Show my Argustack workspace configuration"

8. Raw SQL Queries

Tools `query_issues`, `query_commits`, and `query_prs` accept a `sql` parameter for advanced queries.

Examples

Issues created per month:

```
SELECT date_trunc('month', created::timestamp) as month,
       COUNT(*) as count
FROM issues
GROUP BY month
ORDER BY month DESC
```

Commits without linked issues:

```
SELECT c.hash, c.message, c.author, c.committed_at
FROM commits c
LEFT JOIN commit_issue_refs r ON c.hash = r.commit_hash
WHERE r.commit_hash IS NULL
ORDER BY c.committed_at DESC
LIMIT 20
```

PRs with most review comments:

```
SELECT pr.number, pr.title, pr.author,
       COUNT(pc.id) as comment_count
FROM pull_requests pr
JOIN pr_comments pc ON pr.number = pc.pr_number
GROUP BY pr.number, pr.title, pr.author
ORDER BY comment_count DESC
LIMIT 10
```

Busiest developers (cross-source):

```
SELECT assignee as developer,
       COUNT(*) as issues_resolved,
       (SELECT COUNT(*) FROM commits WHERE author ILIKE '%' || i.assignee || '%') as c
FROM issues i
WHERE status_category = 'Done'
GROUP BY assignee
ORDER BY issues_resolved DESC
LIMIT 10
```

9. Database Schema Reference

All tables available for raw SQL queries:

Jira Tables

Table	Key Columns
`issues`	issue_key, summary, status, assignee, issue_type, components, labels, custom_fields (JSONB), raw_json (JSONB), embedding (vector)
`issue_comments`	issue_key, author, body, created
`issue_changelogs`	issue_key, field, from_value, to_value, changed_at
`issue_worklogs`	issue_key, author, time_spent_seconds, started
`issue_links`	source_key, target_key, link_type

Git Tables

Table	Key Columns
`commits`	hash, message, author, committed_at, repo_path, search_vector
`commit_files`	commit_hash, file_path, status, additions, deletions
`commit_issue_refs`	commit_hash, issue_key

GitHub Tables

Table	Key Columns
`pull_requests`	number, title, state, author, merged_at, base_ref, search_vector
`pr_reviews`	pr_number, reviewer, state
`pr_comments`	pr_number, author, body, path, line
`pr_files`	pr_number, file_path, additions, deletions
`pr_issue_refs`	pr_number, issue_key
`releases`	tag_name, name, body, published_at, author

External Database Tables

Table	Key Columns
`db_tables`	source_name, table_schema, table_name, row_count, size_bytes
`db_columns`	source_name, table_name, column_name, data_type, is_nullable, is_primary_key
`db_foreign_keys`	source_name, table_name, column_name, referenced_table, referenced_column
`db_indexes`	source_name, table_name, index_name, columns (text[]), is_unique, is_primary

Visit the project on **GitHub** or install from **npm**.